

SaveFunds

Analyse technique des services backend

Architecture, logique métier et implémentation Java

Profacile SRL — Juin 2026 — TFE Bachelier Informatique de Gestion

1. Introduction et architecture générale

SaveFunds est une application Spring Boot 3.4 / Java 21 qui permet à un dirigeant de PME belge de simuler la faisabilité financière d'un prélèvement personnel. Le backend est organisé en couches distinctes : contrôleurs REST, services métier, et repositories JPA.

La couche service est le cœur de l'application. Elle contient toute la logique métier, isolée des détails techniques (HTTP, base de données). Cette séparation est un principe fondamental de l'architecture en couches (Layered Architecture).

Service	Responsabilité	Dépendances
AuthServiceImpl	Inscription et connexion des utilisateurs, génération JWT	UserRepository, PasswordEncoder, JwtUtil
EntrepriseServiceImpl	CRUD entreprise, règle 1 user = 1 entreprise, gestion date CC débiteur	EntrepriseRepository
IndicateurServiceImpl	Calcul des 4 indicateurs financiers (trésorerie, ratio, CC, montant max)	Aucune (calculs purs)
GrilleTricoloreServiceImpl	Application des seuils VERT/ORANGE/ROUGE et décision globale	Aucune (logique de décision pure)
AnalyseServiceImpl	Orchestration complète du processus d'analyse en 8 étapes	IndicateurService, GrilleTricoloreService, repositories
ResultatAnalyseServiceImpl	CRUD des résultats d'analyse, unicité 1 analyse = 1 résultat	ResultatAnalyseRepository
SituationFinanciereServiceImpl	Historique des snapshots financiers d'une entreprise	SituationFinanciereRepository

2. AuthServiceImpl — Authentification et sécurité

2.1 Ce qu'on voulait faire

Permettre à un dirigeant de créer un compte et de se connecter de façon sécurisée. Chaque session doit être identifiée par un token JWT (JSON Web Token) qui accompagne toutes les requêtes ultérieures.

2.2 Comment c'est implémenté

Inscription (register)

Avant de créer l'utilisateur, le service vérifie que l'email n'existe pas déjà en base :

```
if (userRepository.existsByEmail(request.getEmail())) {  
    throw new IllegalArgumentException("Email déjà utilisé");  
}
```

Le mot de passe n'est jamais stocké en clair. Il est haché (chiffré de façon irréversible) via BCrypt, un algorithme standard recommandé par l'OWASP :

```
user.setPasswordHash(passwordEncoder.encode(request.getPassword()));
```

Après sauvegarde, un token JWT est généré avec JwtUtil et retourné au client. Ce token contient l'email de l'utilisateur et expire après 24h (configurable dans application.properties).

💡 *Choix technique : BCrypt est un hachage adaptatif — il intègre un 'sel' aléatoire qui empêche les attaques par dictionnaire. Deux mots de passe identiques donnent des hachages différents.*

Connexion (login)

Le service récupère l'utilisateur par email, puis compare le mot de passe fourni avec le hachage stocké :

```
if (!passwordEncoder.matches(request.getPassword(),
    user.getPasswordHash())) {
    throw new IllegalArgumentException("Email ou mot de passe incorrect");
}
```

Le message d'erreur est volontairement identique pour email inexistant et mot de passe incorrect. C'est une bonne pratique de sécurité : éviter de révéler si un email est enregistré ou non dans le système.

💡 *Principe de sécurité : réponse identique pour les deux cas d'échec = protection contre l'énumération de comptes (user enumeration attack).*

3. EntrepriseServiceImpl — Gestion des entreprises

3.1 Ce qu'on voulait faire

Gérer les informations financières d'une entreprise avec une règle métier clé : un utilisateur ne peut avoir qu'une seule entreprise. En plus du CRUD classique, le service doit gérer automatiquement la date de début du compte courant débiteur.

3.2 Comment c'est implémenté

Règle 1 user = 1 entreprise

Lors de la création, avant la sauvegarde, le service vérifie si l'utilisateur possède déjà une entreprise :

```
Optional<Entreprise> existing =
entrepriseRepository.findByUserId(entreprise.getUserId());
if (existing.isPresent()) {
    throw new IllegalArgumentException("L'utilisateur possède déjà
une entreprise...");
}
```

💡 Cette règle est cohérente avec le modèle métier de SaveFunds : un dirigeant de PME gère une seule structure. Elle simplifie aussi l'interface utilisateur (pas de sélecteur d'entreprise).

Mise à jour field-by-field

Le service n'écrase jamais tous les champs — il met à jour uniquement les champs non-null fournis dans la requête. Cela permet des mises à jour partielles (PATCH sémantique) sans perdre les données existantes :

```
if (entreprise.getRaisonSociale() != null) {
    existing.setRaisonSociale(entreprise.getRaisonSociale());
}
```

Gestion automatique de la date du compte courant débiteur

C'est la logique la plus complexe de ce service. Lorsque le solde du compte courant change, le service doit déterminer automatiquement la date de début du débit sans intervention de l'utilisateur :

```
if (entreprise.getSoldeCompteCourant().compareTo(BigDecimal.ZERO) >=
0) {
    // Solde redevenu positif : on réinitialise la date
    existing.setDateDebutDebiteurCC(null);
} else {
    // Solde négatif : on enregistre la date du PREMIER jour de débit
    if (existing.getDateDebutDebiteurCC() == null) {
        existing.setDateDebutDebiteurCC(LocalDate.now());
    }
}
```

La condition 'if (existing.getDateDebutDebiteurCC() == null)' est cruciale : si la date est déjà renseignée (débit en cours), on ne la réinitialise pas. Cela permet de conserver la vraie date de début du débit, même si le dirigeant met à jour d'autres données financières.

💡 Exemple concret : le compte est débiteur depuis le 1er juin. Le 15 juin, le dirigeant met à jour son CA. La date du 1er juin doit être conservée — c'est ce que cette condition garantit.

4. IndicateurServiceImpl — Calcul des indicateurs financiers

4.1 Ce qu'on voulait faire

Calculer les 4 indicateurs financiers bruts à partir des données de l'entreprise. Ce service ne prend aucune décision : il produit uniquement des valeurs numériques. La séparation calcul / décision est un choix architectural délibéré (principe de responsabilité unique — SRP).

4.2 Indicateur 1 — Trésorerie en mois de charges

Formule : trésorerie disponible ÷ charges mensuelles

```
BigDecimal mois = tresorerie.divide(charges, PRECISION_MOIS,
    RoundingMode.HALF_UP);
```

L'utilisation de BigDecimal (et non double ou float) est obligatoire pour les calculs monétaires en Java. Les types flottants binaires introduisent des erreurs d'arrondi inacceptables dans un contexte financier. L'arrondi HALF_UP (arrondi mathématique classique) est utilisé systématiquement.

Si la trésorerie est négative, le service retourne 0 — l'entreprise n'a aucune réserve :

```
if (tresorerie.compareTo(BigDecimal.ZERO) < 0) { return
    BigDecimal.ZERO; }
```

4.3 Indicateur 2 — Ratio CA/Charges

Formule : chiffre d'affaires mensuel ÷ charges mensuelles

```
BigDecimal ratio = chiffreAffaires.divide(charges, PRECISION_RATIO,
    RoundingMode.HALF_UP);
```

Ce ratio mesure la rentabilité opérationnelle. Un ratio de 1.0 signifie que l'entreprise couvre exactement ses charges (seuil de rentabilité). En dessous de 1.0, elle est déficitaire. La validation empêche une division par zéro :

```
if (charges.compareTo(BigDecimal.ZERO) <= 0) { throw new
    IllegalArgumentException(...); }
```

4.4 Indicateur 3 — Durée du compte courant débiteur

Cet indicateur utilise la date enregistrée par EntrepriseServiceImpl pour calculer le nombre de jours réels de débit :

```
long jours = ChronoUnit.DAYS.between(dateDebutDebiteur,
    LocalDate.now());
int duree = (int) Math.max(1, jours); // Minimum 1 jour si solde
négatif
```

La valeur minimale de 1 jour (Math.max(1, jours)) évite le cas limite où le dirigeant met en débit et lance une analyse le même jour — ce qui retournerait 0 jour et masquerait le problème.

Si le solde est négatif mais qu'aucune date n'est enregistrée (données historiques sans date), le service retourne 31 jours par défaut pour déclencher une alerte ROUGE :

```
if (dateDebutDebiteur == null) { return 31; } // → ROUGE par
sécurité
```

4.5 Indicateur 4 — Montant maximum prélevable

Formule : trésorerie – (charges mensuelles × 3 mois de sécurité)

```
BigDecimal tresorerieSeuil =  
chargesMensuelles.multiply(BigDecimal.valueOf(seuilSecurite));  
BigDecimal montantMax = tresorerie.subtract(tresorerieSeuil);
```

Le seuil de 3 mois de sécurité est configurable (paramètre `seuilSecurite`). Si le résultat est négatif, le service retourne 0 — aucun prélèvement n'est possible sans entamer la réserve de sécurité.

💡 La méthode `calculerTousIndicateurs()` regroupe tous les calculs en une seule `Map<String, Object>`. Cela permet à `AnalyseServiceImpl` de récupérer tous les indicateurs en un seul appel, optimisant les performances.

5. GrilleTricoloreServiceImpl — Application des seuils de décision

5.1 Ce qu'on voulait faire

Transformer les valeurs numériques calculées par `IndicateurService` en décisions VERT/ORANGE/ROUGE. Ce service applique les seuils définis dans le document de justification de la grille tricolore.

5.2 Structure des décisions

Chaque méthode de décision suit la même structure : comparaison du score avec les seuils dans l'ordre décroissant (du plus favorable au moins favorable) :

```
if (tresorerieEnMois.compareTo(SEUIL_TRESORERIE_VERT) >= 0) return
Decision.VERT;
if (tresorerieEnMois.compareTo(SEUIL_TRESORERIE_ORANGE) >= 0) return
Decision.ORANGE;
return Decision.ROUGE;
```

Les seuils sont définis comme des constantes statiques en haut de la classe. Ce choix facilite la maintenance : pour modifier un seuil, il suffit de changer une constante, pas de chercher dans le code.

💡 Utilisation de `BigDecimal.compareTo()` et non de l'opérateur `==` ou `equals()` pour comparer des `BigDecimal`. La méthode `equals()` tient compte de la précision ($1.0 \neq 1.00$ avec `equals`), ce qui causerait des bugs. `compareTo()` compare uniquement la valeur numérique.

5.3 Décision globale — règle du maillon faible

La décision globale applique le principe : le critère le plus défavorable l'emporte. Techniquement, les 4 décisions sont passées en paramètre et le service cherche d'abord un ROUGE, puis un ORANGE :

```
if (tresorerie == Decision.ROUGE || ratio == Decision.ROUGE
    || compteCourant == Decision.ROUGE || montant ==
Decision.ROUGE) {
    return Decision.ROUGE;
}
if (tresorerie == Decision.ORANGE || ratio == Decision.ORANGE
    || compteCourant == Decision.ORANGE || montant ==
Decision.ORANGE) {
    return Decision.ORANGE;
}
return Decision.VERT;
```

Pour les énumérations Java (enum), l'opérateur `==` est correct et préférable à `equals()`. Les valeurs d'un enum sont des singletons — il n'existe qu'une seule instance de `Decision.ROUGE` dans la JVM.

5.4 Génération des recommandations

Le service génère des textes d'explication adaptés à chaque combinaison décision/critère. Un switch expression Java 14+ est utilisé pour sa concision et son exhaustivité garantie à la compilation :

```
return switch (critere.toLowerCase()) {
```

```
    case "tresorerie" -> genererRecommandationTresorerie(decision);  
    case "ratio" -> genererRecommandationRatio(decision);  
    case "compte_courant" ->  
genererRecommandationCompteCourant(decision);  
    default -> "Critère inconnu: " + critere;  
};
```

💡 Le *switch expression* (avec `->`) garantit qu'on ne peut pas oublier de traiter un cas. Si on ajoute une valeur à l'enum *Decision* sans mettre à jour les *switch*, le compilateur signale une erreur.

6. AnalyseServiceImpl — Orchestration du processus d'analyse

6.1 Ce qu'on voulait faire

C'est le service central de SaveFunds. Il orchestre l'ensemble du processus d'analyse en 8 étapes séquentielles, depuis la récupération des données jusqu'à la sauvegarde du résultat. Il délègue les calculs à IndicateurService et les décisions à GrilleTricoloreService.

6.2 Les 8 étapes de effectuerAnalyse()

Étape	Opération	Description technique
1	Récupération des données	Chargement de l'analyse et de l'entreprise depuis la base. Vérification que l'analyse n'est pas déjà TERMINEE (idempotence).
2	Vérification des données	Appel à <code>indicateurService.donneesCompletesPresentes()</code> : vérifie que CA, charges, trésorerie et solde CC sont tous renseignés et positifs.
3	Calcul des indicateurs	Appel à <code>indicateurService.calculerTousIndicateurs()</code> qui retourne une Map avec les 4 valeurs numériques calculées.
3.5	Vérification du montant	Comparaison du montant souhaité au montant maximum prélevable. Calcul du ratio (montant / max) pour distinguer VERT (<75%), ORANGE (75-100%) et ROUGE (>100%).
4	Calcul des décisions	Les 4 valeurs numériques + <code>decisionMontant</code> sont passées à <code>grilleTricoloreService</code> pour obtenir 4 décisions individuelles + 1 décision globale.
5	Génération des recommandations	Pour chaque critère, <code>grilleTricoloreService.genererRecommandation()</code> produit un texte d'explication personnalisé.
6	Génération des détails	Des méthodes privées (<code>genererDetailsTrésorerie</code> , <code>genererDetailsRatio</code> , etc.) construisent des phrases explicatives incluant les valeurs calculées.
7	Création du résultat	Un objet <code>ResultatAnalyse</code> est construit via le Builder Pattern (Lombok <code>@Builder</code>) et sauvegardé en base.
8	Mise à jour du statut	L'analyse passe de <code>EN_ATTENTE</code> à <code>TERMINEE</code> . Empêche de relancer le calcul sur la même analyse (garantie d'idempotence).

6.3 La logique de décision sur le montant (étape 3.5)

Cette étape est un ajout majeur par rapport à la version initiale. Le montant souhaité était ignoré dans le calcul original — ce bug a été corrigé. Voici la logique :

```
if (montantMax.compareTo(BigDecimal.ZERO) <= 0) {
    decisionMontant = Decision.ROUGE; // Rien n'est prélevable
} else if (montantSouhaite.compareTo(montantMax) > 0) {
    decisionMontant = Decision.ROUGE; // Montant demandé > maximum
} else {
```

```

        BigDecimal ratio = montantSouhaite.divide(montantMax, 2,
RoundingMode.HALF_UP);
        decisionMontant = ratio.compareTo(new BigDecimal("0.75")) > 0
            ? Decision.ORANGE // Entre 75% et 100% du max
            : Decision.VERT; // Inférieur à 75% du max
    }

```

💡 Le seuil de 75% introduit une zone d'avertissement ORANGE : techniquement possible mais imprudent. C'est une application du principe de prudence comptable belge.

6.4 Pattern Builder pour ResultatAnalyse

L'objet ResultatAnalyse est construit via le Builder Pattern, généré automatiquement par l'annotation @Builder de Lombok. Ce pattern améliore la lisibilité lors de la création d'objets avec de nombreux champs :

```

ResultatAnalyse resultat = ResultatAnalyse.builder()
    .analyse(analyse)
    .decisionGlobale(decisionGlobale)
    .montantMaxPrelevable(montantMax)
    // ... 15 autres champs
    .build();

```

💡 Sans le Builder Pattern, il faudrait un constructeur avec 18 paramètres ou 18 appels setter successifs. Le Builder améliore la lisibilité et réduit les risques d'erreur dans l'ordre des paramètres.

7. ResultatAnalyseServiceImpl — Gestion des résultats

7.1 Ce qu'on voulait faire

Gérer la persistance des résultats d'analyse avec une contrainte forte : une analyse ne peut avoir qu'un seul résultat. Le service doit garantir cette unicité.

7.2 Garantie d'unicité

Avant toute création, le service vérifie qu'il n'existe pas déjà un résultat pour cette analyse :

```
Optional<ResultatAnalyse> existing =
    resultatRepository.findByAnalyse_Id(resultat.getAnalyse().getId());
if (existing.isPresent()) {
    throw new IllegalArgumentException("Un résultat existe déjà pour
    l'analyse ID : ...");
}
```

Cette vérification en service complète la contrainte de base de données (contrainte UNIQUE sur analyse_id dans la table resultats_analyse). La double protection évite les cas de concurrence (race conditions) dans un environnement multi-utilisateurs.

7.3 Mise à jour field-by-field

Comme pour EntrepriseServiceImpl, la mise à jour est partielle : seuls les champs non-null sont mis à jour. Cette approche est cohérente dans tout le projet et permet des mises à jour ciblées sans écraser les données existantes.

8. SituationFinanciereServiceImpl — Historique financier

8.1 Ce qu'on voulait faire

Conserver un historique des snapshots financiers d'une entreprise. Chaque fois que les données financières changent, une situation peut être capturée pour permettre le suivi de l'évolution dans le temps.

8.2 Implémentation

Le service implémente deux méthodes de lecture complémentaires :

- findByEntrepriseId() : retourne tout l'historique trié du plus récent au plus ancien (pour l'affichage d'une liste).
- findLastByEntrepriseId() : retourne uniquement la situation la plus récente (pour les calculs ou l'affichage du tableau de bord).

La date de capture est initialisée automatiquement si non fournie :

```
if (situation.getCapturedAt() == null) {
    situation.setCapturedAt(LocalDate.now());
}
```

9. Choix techniques transversaux

9.1 Gestion des transactions

Tous les services sont annotés `@Transactional` au niveau de la classe. Les méthodes de lecture sont annotées `@Transactional(readOnly = true)` — cela optimise les performances car Spring n'ouvre pas de transaction d'écriture inutile et peut utiliser des répliques en lecture.

9.2 Injection de dépendances

Le projet utilise l'injection par constructeur (`@RequiredArgsConstructor` de Lombok) plutôt que l'injection par champ (`@Autowired`). C'est la pratique recommandée par Spring : les dépendances sont immutables (`final`), ce qui rend le code plus testable et thread-safe.

9.3 Logging structuré

Chaque service utilise `@Slf4j` (Lombok) pour le logging. Les niveaux sont cohérents :

- **DEBUG** : valeurs intermédiaires de calcul (uniquement visibles en mode debug).
- **INFO** : événements métier importants (création, résultat d'analyse).
- **WARN** : anomalies non bloquantes (données incomplètes).
- **ERROR** : exceptions inattendues avec détail de l'erreur.

9.4 Gestion des erreurs

Trois types d'exceptions sont utilisés de façon cohérente :

- `IllegalArgumentException` : données invalides fournies par le client (400 Bad Request).
- `IllegalStateException` : état métier incohérent (analyse déjà calculée, données incomplètes).
- `ResourceNotFoundException` : ressource introuvable en base (404 Not Found).

Ces exceptions sont interceptées par `GlobalExceptionHandler` qui les transforme en réponses HTTP appropriées.

10. Conclusion

L'architecture des services de `SaveFunds` applique plusieurs principes du génie logiciel moderne : séparation des responsabilités (chaque service a un rôle précis), injection de dépendances, gestion transactionnelle, et validation à chaque couche.

La correction majeure apportée — intégration du `montantSouhaite` dans le calcul tricolore — illustre l'importance des tests fonctionnels : le bug était présent depuis l'origine mais n'avait été détecté qu'en testant des scénarios réels avec des montants variés.

Le système est conçu pour être évolutif : les seuils de la grille tricolore sont des constantes modifiables, les services sont découplés via des interfaces, et la logique de calcul est isolée dans `IndicateurServiceImpl` pour faciliter les tests unitaires.

Profacile SRL — contact@profacile.be — Juin 2026